# WFLOW-09: Security, Governance & Monitoring

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 9 of 10 | **Created:** January 2026 | **Last Updated:** 11/09/2026

## Production Best Practices

This last core notebook covers workflow security, governance, observability, and operational best practices for running workflows in production.

## Table of Contents


1. Security Best Practices
2. Secrets Management
3. Setting Up Third-Party Connections
4. Access Control (RBAC)
5. Workflow Observability
6. Alerting on Workflows
7. Change Management
8. Operational Runbook
9. Series Summary
10. Congratulations!

# Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with Platform subscription |
| **Permissions** | `automation:workflows:admin` for governance setup (Workflow admin mode). To run the §5 queries — and for the actor of the §6 monitoring workflow — `storage:system:read` (WHERE `storage:event.provider = "AUTOMATION_ENGINE"`) and `storage:buckets:read` (WHERE `storage:table-name = "dt.system.events"`) |
| **Prior Knowledge** | **WFLOW-01** through **WFLOW-08** |

# 1. Security Best Practices

## Secure Workflow Design

| Principle | Implementation |
|---|---|
| **Least privilege** | Connections with minimum required permissions |
| **No hardcoded secrets** | Use secrets vault, never inline credentials |
| **Input validation** | Sanitize all dynamic inputs |
| **Audit logging** | Log all actions for compliance |
| **Fail secure** | Default to safe state on errors |

## Input Sanitization

```
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const ev = (await execution()).params.event;
  // NEVER do this - injection risk
  // const query = `fetch logs | filter content == "${ev['event.name']}"`;

  // SAFE: Validate and sanitize inputs
  const title = ev['event.name'];

  // Remove potentially dangerous characters
  const sanitized = title
    .replace(/["'\\]/g, '')  // Remove quotes and backslashes
    .substring(0, 200);        // Limit length

  // Use parameterized queries when possible
  return { sanitized_title: sanitized };
}
```

### Secure HTTP Requests

```
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-
environment-v2';

export default async function () {
  const token = (await credentialVaultClient.getCredentialsDetails({ id:
'CREDENTIALS_VAULT-XXXXXXXXXXX' })).token;
  // NEVER log or expose secrets
  // console.log(token);  // BAD!

  // ALWAYS use HTTPS
  const response = await fetch('https://api.example.com/endpoint', {  // NOT
http://
    headers: {
      'Authorization': `Bearer ${token}`  // Token from the Credential Vault
    }
  });

  // DON'T return sensitive data
  return {
    status: response.status,
    success: response.ok
    // NOT: response_body: await response.text()  // May contain secrets
  };
}
```

## 2. Secrets Management

### Where Secrets Live

The Workflows docs describe no `env` secrets object and no per-workflow secrets page. An earlier revision of this notebook used `{{ env.SECRET_NAME }}`; the expression reference lists no `env`, so those expressions do not resolve. Secrets live in one of two places:

| Where | Used by | How |
|---|---|---|
| **Connections** | Connector actions (Slack, Teams, ServiceNow, Jira, PagerDuty, …) | Select the connection in the task; the action reads the credential |

| Where | Used by | How |
| --- | --- | --- |
| **Credential Vault** | HTTP Request action; Run JavaScript | HTTP Request: the task's **Authentication** field (Basic or Token). JavaScript: `credentialVaultClient` |

**Creating a Credential Vault entry for workflows:**

1. Open **Credential Vault** and add a credential (token, or username and password)
2. Set the scope to **AppEngine**
3. For JavaScript use, turn on **Allow access without app context**
4. Give the workflow actor access to the credential
5. Reference it by its ID (`CREDENTIALS_VAULT-…`)

## Secrets in JavaScript

```
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-
environment-v2';

export default async function () {
  const credential = await credentialVaultClient.getCredentialsDetails({ id:
'CREDENTIALS_VAULT-XXXXXXXXXXX' });
  const apiToken = credential.token;

  if (!apiToken) {
    throw new Error('Credential CREDENTIALS_VAULT-XXXXXXXXXXX has no token');
  }

  // Use apiToken in a request header. Never log it or return it:
  // execution results are visible to anyone with read access to the workflow.
  return { configured: true };
}
```

## Secrets in HTTP Request Tasks

Do not put a secret in a header value. The HTTP Request action docs: *"We strictly advise against providing any static Authorization header and therefore, leak a secret. Use the credential vault to store your credentials for Basic or Token authentication, or a Run JavaScript action to implement any other authentication."* Select the credential in the task's **Authentication** field instead:

```
input:
  url: "https://api.example.com/webhook"
  method: POST
  # Authentication: Token -&gt; CREDENTIALS_VAULT-XXXXXXXXXXX (task's
Authentication field)
  headers:
    Content-Type: "application/json"
```

### Secret Rotation

1. Update the credential in the Credential Vault (or the connection)
2. Test workflow execution
3. Revoke the old token at its source

> **Sources:** Run JavaScript action (DT docs) — `credentialVaultClient` and the required credential settings. HTTP request action (DT docs), Jinja expressions for Workflows (DT docs).

## 3. Setting Up Third-Party Connections

WFLOW-03 introduces the Dynatrace **Connection** abstraction; WFLOW-05 walks through PagerDuty and ServiceNow workflow tasks. This section closes the remaining gap: **how to mint the actual vendor-side credentials** that those Connections wrap, with current vendor scope/auth model details and rotation patterns that don't break running workflows.

### 3.1 Slack — App + OAuth Token

The Dynatrace Slack action authenticates as a Slack app (bot token), not as a user. Steps:

0. **Prepare Dynatrace** — add the Slack domain as a host pattern under **Settings > General > External requests**, and in **Workflows > Settings > Authorization settings** grant the permissions the Slack Connector setup page lists ( `app-settings:objects:read` , `app-settings:objects:write` and the `state:*` permissions).
1. **Create the app** at `https://api.slack.com/apps` → **From scratch** → name it (e.g. *Dynatrace Alerts*) and pick the target workspace.

2. **Add OAuth scopes** under **OAuth & Permissions → Bot Token Scopes**. Dynatrace marks two scopes as required for *Send message*, and publishes a minimal manifest with exactly `channels:read`, `groups:read` and `chat:write`:
   - `chat:write` (required) — *"Send messages as your Slack app"* (replaces legacy `chat:write:user` / `chat:write:bot`).
   - `channels:read` (required) — the action needs it to look public channels up by name for the channel selection.
   - `groups:read` (optional) — the same lookup for private channels.
   - `chat:write.public` / `channels:join` (optional) — post to, or join, a public channel the bot is not a member of. `chat:write.public` must be requested alongside `chat:write`.
3. **Install to workspace** — the same page. Slack issues a **Bot User OAuth Token** (the workspace-scoped token a workflow uses). Bot tokens conventionally begin with `xoxb-`; verify the prefix on issue and store the literal value, not the prefix family.
4. **Create the Dynatrace Connection** — **Settings → Connections → Slack → Connection**, give it a descriptive name (e.g. `slack-prod-alerts`) and paste the token into **Bot token**.

> **Sources:** Slack Connector actions (DT docs) — *"channels:read Looking up public channels by name for the channel selection. Required"*; Set up Slack Connector (DT docs) — *"Return to Dynatrace, go to Settings , and select Connections > Slack ."*
>
> **Slack docs:** chat.write scope (docs.slack.dev) — *"Send messages as your Slack app"*; chat.write.public scope (docs.slack.dev) — allows posting to channels the app isn't a member of, requires `chat:write` too.

**Token type:** the Slack connection takes a bot token (OAuth app, the path above); it has no webhook option. An incoming webhook can only be called from an HTTP Request task, and it is pinned to one channel and loses the ability to update / delete / thread messages, react with emoji, or use Block Kit interactivity. WFLOW-03 documents the webhook variant for the minimal case; production Slack integrations should use the OAuth app.

## 3.2 PagerDuty — Events API v2 Integration Key vs REST API Key

PagerDuty exposes two distinct credentials for two distinct surfaces. Picking the wrong one is the single most common WFLOW-05 setup mistake.

| Credential | Where minted | What it does | Use for |
|---|---|---|---|
| **Integration Key** (a.k.a. routing key) | On a single PagerDuty *service* → *Integrations* tab → *Add an integration* → **Events API v2** | Sends *events* (trigger / acknowledge / resolve) to the one service it belongs to. No account-level read access. | Routing Dynatrace problems / Davis events into PagerDuty as incidents. The default WFLOW-05 happy path. |
| **REST API Key** | Account-level → *Integrations* → *API Access Keys* (general access) or *User* → *My Profile* → *User Settings* → *Create API User Token* (personal) | Account-scoped CRUD across services, schedules, escalation policies, incidents, users. | Querying on-call schedules, updating incident status from a workflow, listing services — workflow tasks that need *more than just "open an incident."* |

PagerDuty's own integration guidance draws the same boundary — Events API v2 is intended for *machine-generated monitoring and event data*, REST API for *human-generated events, tickets or incidents* requiring richer CRUD. The Dynatrace `dynatrace.pagerduty:*` action set covers both; pick the credential type matching the action's verb (trigger/ack/resolve → Integration Key; query/update incident metadata → REST API key).

**dedup_key (Events API v2):** each Events API payload carries an optional `dedup_key`. PagerDuty correlates subsequent `trigger`, `acknowledge`, and `resolve` events with the same `dedup_key` against the **same open incident**, so a Davis problem can drive the full lifecycle: trigger on `ACTIVE`, resolve on `CLOSED`, all referencing the same key. WFLOW-05 §7 already recommends `dynatrace-{problem_id}` — keep that pattern: it makes the Dynatrace problem ID the join key on both sides.

> **PagerDuty docs:** Services and integrations (PagerDuty Support) — *"Events API v2 is designed to handle machine-generated monitoring and event data"* and *"For human-generated events, tickets, or incidents, such as those from ServiceNow or JIRA, use the REST API to enable direct, streamlined creation of PagerDuty incidents."*; Events API v2 overview (PagerDuty Developer).

## 3.3 ServiceNow — OAuth Client Credentials vs Basic Auth

The Dynatrace ServiceNow connection offers exactly two authentication types: **Basic Authentication** (username and password of an integration user) and **OAuth Client Credentials** (client ID and client secret). Other OAuth grants — resource-owner password, authorization code, JWT bearer — cannot be configured on the connection, whatever the ServiceNow instance itself accepts.

| Type | When to use for Dynatrace Workflows |
|---|---|
| **OAuth Client Credentials** | Preferred where your instance supports it: no integration-user password is stored in Dynatrace, and the secret can be rotated on the ServiceNow side. One client ID / secret pair per environment (prod/staging). |
| **Basic Authentication** | The path Dynatrace's own setup steps walk through. Use a dedicated integration user, never a person's account. |

In community practice, security teams push toward OAuth by policy — many forbid storing integration-user passwords in third-party systems — rather than because the platform requires it. Mint the OAuth client in your instance's OAuth application registry; the exact menu path and token lifetime defaults depend on your ServiceNow release, so follow your instance's documentation.

**Permissions for the integration user.** Dynatrace documents the permissions as table access rather than a role name. The user needs to:

- search, create and update incidents (table `incident`);
- read categories and subcategories (table `sys_choice`, elements `category` / `subcategory`);
- read assignment groups (table `sys_user_group`);
- read resolution codes (table `sys_choice`, element `close_code`).

Grant these through a narrow custom role and *avoid* `admin` — WFLOW-05 §4 already calls out least privilege.

**Dynatrace side.** Create the connection under **Settings → Connections → ServiceNow** (one per ServiceNow environment), and grant `app-settings:objects:read` in **Workflows → Settings → Authorization settings**.

> **Sources:** Set up ServiceNow Connector (DT docs) — *"Either use basic authentication or OAuth client credentials. For Basic Authentication , provide your username and password. For OAuth Client*

> *Credentials , provide your client id and client secret."* and *"Search, create and update incidents (table incident )"*.

## 3.4 Rotation Patterns Without Breaking In-Flight Workflows

The instinct on rotation day is *"update the secret value and save."* That works on a quiet system; on one with running workflows, community practice is to avoid it — a task that starts while the old credential is already revoked fails. The safer pattern is **dual-credential overlap**:

1. **Mint the new credential** at the vendor (new Slack bot token, new PagerDuty Integration Key on the same service, new ServiceNow OAuth client). Both old and new are now active.
2. **Create a parallel Dynatrace Connection** with a `-v2` suffix (e.g. `slack-prod-alerts-v2`). Don't overwrite the existing one yet.
3. **Migrate workflows one at a time** — update each workflow's task to point at `-v2`, run an On-Demand execution, verify the result vendor-side, then move to the next workflow.
4. **Wait one full execution cycle** beyond the slowest scheduled workflow (typically 24–48 hours for daily reports; longer for weekly) to confirm no workflow still references the old connection.
5. **Revoke the old credential** at the vendor. Only after revocation is the rotation complete — if the old credential still works, the rotation has only added a credential, not retired one.
6. **Delete the old Dynatrace Connection**.

This pattern adds one day of dual-credential window, but no workflow ever runs against an invalid token. With the naive "edit the secret value in place" path, any task that calls the vendor between revocation and the update fails with the vendor's authentication error.

**When to rotate:**

| Trigger | Cadence |
|---|---|
| Routine | Set by your security policy. In community practice, quarterly for Slack bot tokens and ServiceNow OAuth secrets and annually for PagerDuty Integration Keys is a common baseline — no vendor mandates a cadence. Slack bot tokens do not expire on their own unless token rotation is turned on, which Dynatrace's Slack manifest leaves off. |

| Trigger | Cadence |
| --- | --- |
| Personnel change | When the engineer who originally minted the credential leaves the team or company |
| Suspected exposure | Immediately. Bypass the overlap pattern; revoke first, accept the workflow outage, restore on the new credential |
| Vendor-forced | When Slack / PagerDuty / ServiceNow issues a security advisory or forces a token regeneration |

> **Sources:** Using token rotation (docs.slack.dev) — *"Without token rotation, the access token never expires. With token rotation, it expires every 12 hours."*

## 3.5 Detecting Workflows That Use a Connector App

Before rotating, find every workflow that might use the connection — both to scope the migration and to be sure step 3 of the rotation pattern is exhaustive. Execution events record which connector **app** and function a task called, **not** which connection or what input it used. So the query below scopes the candidates: every workflow that has called the ServiceNow connector in the last 30 days, whichever connection it used. Then confirm the connection on each candidate's task — the task's `connection` input, visible in the editor or in the exported workflow. A workflow that has not run in the window does not appear, so also search the Workflows list for the connector's tasks.

```
// Inventory: which workflows invoke a given connector — replace the literal
below
//
// Data object corrected 08/12/2026 along with the rest of this series:
workflow executions live in
// `dt.system.events` with `event.kind == "WORKFLOW_EVENT"`, not in `events`
under a non-existent
// `automation.task.execution` type. `task.input` is not a field either — the
connector actually
// invoked is `dt.automation_engine.action.app` / `.action.function`, which is
a better key than
// scraping an input payload for a connection name. The connection itself is
NOT recorded.
// Each action writes a RUNNING record and a final record — keep the final
one, so one row is one call.
fetch dt.system.events, from:-30d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter contains(dt.automation_engine.action.app, "servicenow")
| summarize {
    last_execution  = takeMax(timestamp),
    execution_count = count()
  }, by:{dt.automation_engine.workflow.title, dt.automation_engine.task.name,
dt.automation_engine.action.function}
| sort last_execution desc
| limit 25
```

If a workflow shows up here, it calls the connector — check which connection its task uses, and migrate it before revoking the old credential. If a workflow you expected to see *doesn't* show up, it either hasn't executed in 30 days (extend the lookback) or calls the connector only in a code path that hasn't been hit.

**For a wider sweep across all external connections:**

```
// Inventory: which connector apps workflows actually invoke
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {executions = count(), errors = countIf(dt.automation_engine.state == "ERROR"), workflows =
countDistinctExact(dt.automation_engine.workflow.id)}, by:
{dt.automation_engine.action.app}
| fieldsAdd error_pct = round(100.0 * errors / executions, decimals: 1)
| sort executions desc
| limit 25
```

`error_pct` per connector app shows where calls are failing. The reason is in `dt.automation_engine.state_info` on the `ERROR` records — an authentication error

points at the credential, while `Blocked request to '…' (host not in allowlist)` means the target host is missing from **Settings > General > External requests**. Alert on a rising error count per app (§6) rather than waiting for a credential to fail outright.

### 3.6 Cross-References

- **WFLOW-03** (§ Setting Up Connections, § Slack Notifications, § Microsoft Teams Notifications) — minimal Connection-creation walkthrough and webhook-vs-OAuth choice for Slack/Teams.
- **WFLOW-05** (§ PagerDuty Setup, § ServiceNow Setup) — the workflow-task side: how to consume the credentials this section mints. WFLOW-05 §7 covers `dedupKey` and `correlation_id` patterns referenced above.
- **WFLOW-06** (§ Slack Block Kit, § Teams Adaptive Cards) — message-template patterns that depend on the OAuth-app credential path (Block Kit interactivity is unavailable through Incoming Webhooks).

## 4. Access Control (RBAC)

### Workflow Permissions

The IAM reference defines four `automation:workflows:*` permissions. There is no separate delete permission, and no `automation:connections:*` permission:

| Permission | Allows |
|---|---|
| `automation:workflows:read` | View workflows |
| `automation:workflows:write` | Create, update and delete workflows, including their schedule and event-trigger configuration. One condition: `automation:workflow-type` (`SIMPLE` or `STANDARD`; operators `IN`, `=`) |
| `automation:workflows:run` | Run workflows manually via the UI or API |
| `automation:workflows:admin` | Administer workflows — access all workflows and executions in **Workflow admin mode** |

Using Workflows at all also needs `app-engine:apps:run`; writing and running workflows needs `app-engine:functions:run`. Reading execution history needs the two `storage:*` permissions in the Prerequisites.

**Connections are settings objects.** Access to a connector's connections is granted on its settings schema, for example:

```
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaId = "app:dynatrace.pagerduty:connection";
```

## IAM Policy Example

A broad user base that may build only simple workflows (one trigger, one task — they don't consume workflow hours):

```
ALLOW app-engine:apps:run, app-engine:functions:run;
ALLOW automation:workflows:read, automation:workflows:run;
ALLOW automation:workflows:write WHERE automation:workflow-type = "SIMPLE";
```

The IAM reference lists no condition on a workflow's owner. "Own workflows" is enforced by ownership and visibility (below), not by a policy statement.

## Team-Based Access Pattern

| Team | Permissions |
|---|---|
| **Workflow Admins** | All user permissions + `automation:workflows:admin` |
| **SRE Team** | Read, write, run; works on workflows owned by the SRE group |
| **App Teams** | Read, write, run on private workflows owned by the team's group |
| **Viewers** | `automation:workflows:read` on public workflows |

## Actor, Owner and Visibility

- **Actor.** A workflow runs with its actor's permissions. Dynatrace highly recommends a **service user** as the actor for production workflows that several people work on. Editing a workflow makes the editor its actor — unless the actor is a service user or the edit is made in Workflow admin mode. Selecting a service user as actor needs `iam:service-users:use`:

`ALLOW iam:service-users:use WHERE iam:service-user-email IN ("<SERVICE_USER_EMAIL>");`

- **Owner and visibility.** A new workflow is **private** — only its owner can view, manage and run it. The owner can make it public (visible to every user with `automation:workflows:*` permissions) or transfer ownership to another user or to a **group**, so every member of the group can access it, subject to their permissions.
- **Workflow admin mode.** Users with `automation:workflows:admin` turn it on under **Settings** in the Workflows app to manage workflows whose owner is unavailable, and to import or edit workflows while preserving their actor and owner.

## Workflow Ownership in Exported Workflows

Ownership lives in the workflow's own fields. There is no free-form ownership metadata block — an exported workflow's `metadata` holds only the version and app dependencies. Record team and classification in the `description` or the naming convention (§7):

```
workflow:
  title: sre-problem-notifications-prod
  description: "team=platform; classification=production"
  owner: &amp;lt;group-uuid&amp;gt;          # the owning group
  ownerType: GROUP             # USER or GROUP
  isPrivate: true
  actor: &amp;lt;service-user-id&amp;gt;     # service user, not a person
```

> **Sources:** IAM policy statements (DT docs) — *"automation : workflows:write Grants permission to write workflows conditions: automation:workflow-type - A string that identifies a workflow type either SIMPLE or STANDARD operators: IN , ="*; Workflow security (DT docs) — *"automation:workflows:write Write workflows. It includes creating, updating, and deleting a workflow."*, *"We highly recommend using service users as actors for all workflows that are worked on collaboratively and serve a production grade use case."*, *"A user who updates a workflow is set as the actor automatically."* and *"Right after a workflow is created, only the owner can view, manage, and execute the workflow."*; PagerDuty Connector setup (DT docs) — *"ALLOW settings:objects:read, settings:objects:write, settings:schemas:read WHERE settings:schemaId = "app:dynatrace.pagerduty:connection""*; client-automation SDK reference (Dynatrace Developer) — workflow fields `actor`, `owner`, `ownerType`, `isPrivate`.

# 5. Workflow Observability

## Key Metrics to Monitor

| Metric | Query | Alert Threshold |
|---|---|---|
| Execution success rate | See cell below | < 95% |
| Execution duration | See cell below | > 5 min avg |
| Failed executions | See cell below | > 5 per hour |
| Throttled workflows | Workflows overview → throttled filter (1,000 event-triggered executions / hour / workflow) | Any |

## Execution History Dashboard

Build a dashboard with these queries to monitor workflow health.

```
// Overall workflow health - last 24 hours
// Corrected 09/24/2026: there is no `automation.workflow.execution` event type, so the earlier
// version of this cell returned 0 for every count. Executions are in `dt.system.events`.
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT" and event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state == "SUCCESS"), failed = countIf(dt.automation_engine.state == "ERROR")}
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
```

```
// Per-workflow success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state == "SUCCESS"), failed = countIf(dt.automation_engine.state == "ERROR")}, by: {dt.automation_engine.workflow.title}
| fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
| sort failed desc
| limit 25
```

```
// Execution trend over time
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| makeTimeseries executions = count(), interval:1h, by: {dt.automation_engine.state}
```

```
// Recent failures with error details
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title, dt.automation_engine.state_info, dt.automation_engine.workflow_execution.id
| sort timestamp desc
| limit 25
```

```
// Task-level performance
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
// Durations: SUCCESS and ERROR only. DISCARDED and SKIPPED are final states too, but those tasks
// never ran and always carry duration 0 (38% of final task records over 24 h on the validation tenant, 10/06/2026).
| filter in(dt.automation_engine.state, {"SUCCESS", "ERROR"})
| summarize {executions = count(), avg_ms = avg(duration) / 1ms}, by: {dt.automation_engine.workflow.title, dt.automation_engine.task.name}
| sort avg_ms desc
| limit 25
```

## 5.1 Deeper debugging patterns

The queries above give a workflow-level health view. The patterns below drill into **per-task execution detail** — what readers actually need when a workflow execution ends in `ERROR` and the high-level dashboard doesn't reveal which task in which step broke.

Workflow executions are recorded in `dt.system.events` with `event.kind == "WORKFLOW_EVENT"`, split by `event.type`:

| `event.type` | One per | Key fields |
|---|---|---|
| `WORKFLOW_EXECUTION` | Workflow run | `dt.automation_engine.workflow_execution.id`, `dt.automation_engine.state`, `dt.automation_engine.state_info`, `duration`, `dt.automation_engine.workflow.title` |
| `TASK_EXECUTION` | Task run within a workflow | `dt.automation_engine.workflow_execution.id` (parent run), `dt.automation_engine.task.name`, `dt.automation_engine.state`, `dt.automation_engine.state_info` |
| `ACTION_EXECUTION` | Action call within a task | `dt.automation_engine.action.app`, `dt.automation_engine.action.function`, `dt.automation_engine.state` |

`dt.automation_engine.workflow_execution.id` is the **correlation key** that joins task and action records to their parent workflow run. (The `automation.workflow.execution` / `automation.task.execution` event types named in earlier revisions do not exist.)

> **Sources:** Workflows umbrella (DT docs), Workflow reference (DT docs), Dynatrace Query Language reference (DT docs).

## 5.2 Failed executions with task-level detail

When a workflow ends in `ERROR`, the actionable question is *which task in the run failed and what was the error?* This goes one level below *Recent failures with error details* in §5 (workflow-level failures) by filtering on `TASK_EXECUTION` records and surfacing `dt.automation_engine.state_info`.

```
// Failed task executions in the last 24h, with their workflow
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| summarize failures = count(), by:{dt.automation_engine.workflow.title,
dt.automation_engine.task.name}
| sort failures desc
| limit 25
```

## 5.3 Slowest tasks across all workflows (duration percentiles)

The `avg` in *Task-level performance* (§5) hides long-tail behavior. A task whose p99 is 4× its p50 is a different problem from one with a uniformly high p50 — the first is a tail-latency

/ external-dependency issue, the second is a structural bottleneck. Percentile aggregations make the distinction visible.

```
// Slowest tasks across all workflows (last 7d) by p95
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
// Durations: SUCCESS and ERROR only. DISCARDED and SKIPPED are final states too, but those tasks
// never ran and always carry duration 0 (38% of final task records over 24 h on the validation tenant, 10/06/2026).
| filter in(dt.automation_engine.state, {"SUCCESS", "ERROR"})
| summarize {executions = count(), p95_ms = percentile(duration, 95) / 1ms}, by:{dt.automation_engine.workflow.title, dt.automation_engine.task.name}
| filter executions &gt; 5
| sort p95_ms desc
| limit 20
```

## 5.4 Filter by workflow ID, task name, status

Targeted lookup for a single workflow + task combination. Adapt the literals to the workflow and task you are debugging — useful when a specific notification keeps failing and you need the error history for just that step.

```
// Targeted lookup — replace the literal with a workflow title from your
tenant
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter contains(dt.automation_engine.workflow.title, "Alert")
| fields timestamp, dt.automation_engine.workflow.title,
dt.automation_engine.task.name, dt.automation_engine.state, duration
| sort timestamp desc
| limit 25
```

## 5.5 Workflow failures in the last hour

A short-window failure list, suitable for a dashboard tile next to Davis problems. To follow a failing run into downstream logs, there is no built-in join key: execution records carry

`dt.automation_engine.workflow_execution.id`, but logs do not carry it. Log the execution ID from your JavaScript task (or pass it to the target service) and filter your logs on it.

```
// Workflow failures in the last hour
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-1h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title, dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

### 5.6 Promoting these queries to a dashboard

These queries are written as ad-hoc investigation patterns — copy into a Notebook section, change the time range, run. To make them part of standing operations, promote them to a dashboard:

- **Workflow Health overview** — the §5 overall-health success-rate tile + the §5.3 percentile table + the §5.2 failed-task list.
- **Per-workflow drilldown** — parameterize the §5.4 lookup with a `dt.automation_engine.workflow.title` variable so SREs can pick a workflow from a dropdown.
- **Cross-signal correlation** — the §5.5 failure list next to a Davis Problems tile filtered to the same time window, so failures and incidents appear side-by-side.

Dashboard tile selection, variable binding, and layout patterns live in the **DASH** series — see *DASH-02 (tile types)*, *DASH-04 (variables and parameterization)*, and *DASH-06 (operational dashboards)*. The DQL stays the same; only the surface (Notebook section → dashboard tile) changes.

## 6. Alerting on Workflows

### Create a Workflow-Monitoring Workflow

Use a scheduled workflow to monitor other workflows:

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

async function runQuery(query) {
  // queryExecute returns a requestToken instead of the result when the query
outlasts
  // requestTimeoutMilliseconds - poll until it reaches a final state (see
WFLOW-08 §2).
  const started = await queryExecutionClient.queryExecute({ body: { query,
requestTimeoutMilliseconds: 30000 } });
  let r = started;
  while (r &amp;amp;&amp;amp; (r.state === 'NOT_STARTED' || r.state === 'RUNNING')) {
    r = await queryExecutionClient.queryPoll({ requestToken:
started.requestToken, requestTimeoutMilliseconds: 30000 });
  }
  if (!r || r.state !== 'SUCCEEDED') throw new Error(`DQL query did not
succeed: ${r?.state ?? 'no response'}`);
  return r.result.records;
}

export default async function () {
  // Query for failures in the last hour
  const failingWorkflows = await runQuery(`
    fetch dt.system.events, from:-1h
    | filter event.kind == "WORKFLOW_EVENT" and event.type ==
"WORKFLOW_EXECUTION"
    | filter dt.automation_engine.state.is_final == true
    | summarize failures = countIf(dt.automation_engine.state == "ERROR"), by:
{dt.automation_engine.workflow.title}
    | filter failures &gt;= 3
  `);

  if (failingWorkflows.length &gt; 0) {
    return {
      alert: true,
      failing_workflows: failingWorkflows,
      message: `${failingWorkflows.length} workflows with 3+ failures in the
last hour`
    };
  }

  return { alert: false };
}
```

## Conditional Alert Based on Check

In the workflow document, `tasks` is a map keyed by task name, and each task carries its own `conditions` (`states` of predecessors, an optional `custom` expression, and `else`). The Slack action needs a connection; the `connection()` expression resolves a connection's ID from its name:

```
tasks:
  check_workflows:
    name: check_workflows
    action: dynatrace.automations:run-javascript
    input:
      script: |
        // paste the check script above
  alert_slack:
    name: alert_slack
    action: dynatrace.slack:slack-send-message
    predecessors:
      - check_workflows
    conditions:
      states:
        check_workflows: OK
      custom: '{{ result("check_workflows").alert }}'
      else: SKIP
    input:
      connection: "{{ connection('app:dynatrace.slack:connection', 'slack-
prod-alerts') }}"
      channel: "#workflow-alerts"
      message: |
        :warning: *Workflow Health Alert*

        {{ result("check_workflows").message }}

        {% for wf in result("check_workflows").failing_workflows %}
        • {{ wf["dt.automation_engine.workflow.title"] }}: {{ wf.failures }}
failures
        {% endfor %}
```

The workflow's actor needs the `storage:*` permissions from the Prerequisites, or the check query reads nothing.

> **Sources:** Jinja expressions for Workflows (DT docs) — *"connection() Get a single connection by schema ID and connection name."*; Slack Connector actions (DT docs) — *"Connection Connection ID.*

> *Required"*; threat-detection-notification-sender.yaml (Dynatrace GitHub) — task shape with `conditions: states / custom / else`.

# 7. Change Management

## Version Control

Export workflows for version control with the dedicated export endpoint:

```
# Export via API
curl -X GET
"https://&lt;env&gt;/platform/automation/v1/workflows/&lt;id&gt;/export" \
  -H "Authorization: Bearer &lt;platform-token&gt;" \
  -o workflow-export.json
```

The platform token needs the `automation:workflows:read` scope. Platform tokens are sent as `Bearer`, not `Api-Token` (Platform tokens (DT docs)).

## Workflow Naming Convention

```
&lt;team&gt;-&lt;function&gt;-&lt;environment&gt;

Examples:
- sre-problem-notifications-prod
- checkout-auto-remediation-staging
- platform-daily-report-all
```

## Testing Changes

1. **Clone workflow** to test version
2. **Modify clone** with changes
3. **Test with On-Demand trigger**
4. **Review execution results**
5. **Promote to production workflow**

### Rollback Procedure

1. Disable failing workflow (toggle off)
2. Restore the previous version from the workflow's history — `POST /platform/automation/v1/workflows/<id>/history/<version>/restore` (scope `automation:workflows:write`), which deploys the restored version. Import from your own export only if the history no longer has it.
3. Enable restored workflow
4. Verify execution

> **Sources:** client-automation SDK reference (Dynatrace Developer) — `exportWorkflow`, and `restoreWorkflowHistoryRecord`: *"Restores the workflow to the specified history version which is deployed afterward."*

## 8. Operational Runbook

### Daily Operations

| Task | Frequency | Action |
|---|---|---|
| Check dashboard | Daily | Review execution success rates |
| Review failures | Daily | Investigate and fix any failures |
| Verify connections | Weekly | Test all external connections |
| Rotate secrets | Per your policy (§3.4) | Dual-credential overlap, then revoke the old one |

### Troubleshooting Common Issues

| Symptom | Likely Cause | Resolution |
|---|---|---|
| Task timeout | External API slow | Increase timeout, add retry |

| Symptom | Likely Cause | Resolution |
|---|---|---|
| Auth failure | Revoked or rotated credential | Update the connection (§3.4) |
| Throttled (HTTP 429) | Over 1,000 event-triggered executions / hour | Narrow the trigger filter |
| Blocked request (host not in allowlist) | Target missing from External requests | Add a host pattern |
| Missing data | Query returned empty | Check time range, filters |

## Escalation Path

```
Level 1: Check execution history, review error message
Level 2: Check connections, verify external services
Level 3: Review workflow logic, check for regressions
Level 4: Contact Dynatrace support
```

## Capacity Planning

| Limit | Value | Source |
|---|---|---|
| Task timeout | 60 min default, max 7 days | Build workflows (DT docs) |
| Per-action runtime | 120 s | Build workflows (DT docs) |
| Event-triggered executions | 1,000 / hour / workflow | Upgrade guide (DT docs) |
| Trigger filter expression | 1,000 characters | Event triggers (DT docs) |
| Workflows per environment | 10,000 (100 on trial) | Upgrade guide (DT docs) |

Concurrency and tasks-per-workflow caps are not published; see WFLOW-01 § 5. An earlier revision listed 100 concurrent executions, 15 minutes and 50 tasks with no source.

> **Sources:** Build workflows (DT docs) — *"The default timeout is 60 minutes, up to seven days."* Upgrade guide — alerting and notifications (DT docs) — *"Execution rate: 1,000 event-triggered executions per hour, per workflow."* and *"Environment limits are 10,000 workflows per customer*

> *environment and 100 per trial environment."* Event triggers for workflows (DT docs) — the 1,000-character filter limit.

# 9. Series Summary

## What You've Learned

| Notebook | Key Topics |
|---|---|
| **WFLOW-01** | Workflow fundamentals, components, first workflow |
| **WFLOW-02** | Trigger types: Problem, Davis event, schedule, on-demand, event |
| **WFLOW-03** | Slack, Teams, email notification basics |
| **WFLOW-04** | Conditional routing, escalation patterns |
| **WFLOW-05** | PagerDuty and ServiceNow integration |
| **WFLOW-06** | Custom templates, Jinja, Block Kit, Adaptive Cards |
| **WFLOW-07** | Auto-remediation, guardrails, approval workflows |
| **WFLOW-08** | JavaScript SDK, HTTP requests, custom integrations |
| **WFLOW-09** | Security, governance, monitoring, operations |
| **WFLOW-94 LAB** | EdgeConnect static egress IP for IP-allow-listed targets (Snowflake) |
| **WFLOW-95 LAB** | CMDB-driven host tag enrichment |
| **WFLOW-99** | Best-practice summary across the series |

## Implementation Checklist

- [ ] Create connections for notification channels
- [ ] Build problem notification workflow
- [ ] Configure severity-based routing
- [ ] Integrate incident management (PagerDuty/ServiceNow)
- [ ] Design custom notification templates
- [ ] Implement auto-remediation (if applicable)

- [ ] Set up workflow monitoring dashboard
- [ ] Configure workflow health alerts
- [ ] Document operational procedures
- [ ] Train team on workflow management

### Best Practices Summary

1. **Start simple** - Basic notifications before complex automation
2. **Test thoroughly** - Use On-Demand trigger for testing
3. **Secure by default** - Never hardcode secrets
4. **Monitor everything** - Dashboard for workflow health
5. **Document changes** - Version control and change log
6. **Plan for failure** - Error handling and rollback

---

## Congratulations!

You've completed the core WFLOW series — WFLOW-99 consolidates its best practices, and the 94/95 LABs apply them end to end. You now have the knowledge to:

- Build event-driven notification workflows
- Integrate with incident management platforms
- Create custom automation with JavaScript
- Implement auto-remediation safely
- Operate workflows in production

---

## References


- Workflows umbrella (DT docs)
- Workflow actions umbrella (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Identity and Access Management umbrella (DT docs)
- Manage user permissions / boundaries (DT docs)

- Dynatrace Developer Portal (Dynatrace)
- Slack chat.write scope (docs.slack.dev)
- Slack chat.write.public scope (docs.slack.dev)
- PagerDuty Services and integrations (PagerDuty Support)
- PagerDuty Events API v2 overview (PagerDuty Developer)
- ServiceNow inbound REST API (ServiceNow Docs)

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*